Enabling Uniform Computer Interaction Experience for Blind Users through Large Language Models

Satwik Ram Kodandaram*
Department of Computer Science, Stony Brook University
United States
skodandaram@cs.stonybrook.edu

Utku Uckun*
Computer Science, Stony Brook University
United States
uuckun@cs.stonybrook.edu

Xiaojun Bi
Department of Computer Science, Stony Brook University
United States
xiaojun@cs.stonybrook.edu

IV Ramakrishnan
Computer Science, Stony Brook University
United States
ram@cs.stonybrook.edu

Vikas Ashok
Department of Computer Science, Old Dominion University
United States
vganjigu@odu.edu

Abstract

Blind individuals, who by necessity depend on screen readers to interact with computers, face considerable challenges in navigating the diverse and complex graphical user interfaces of different computer applications. The heterogeneity of various application interfaces often requires blind users to remember different keyboard combinations and navigation methods to use each application effectively. To alleviate this significant interaction burden imposed by heterogeneous application interfaces, we present Savant, a novel assistive technology powered by large language models (LLMs) that allows blind screen reader users to interact uniformly with any application interface through natural language. Novelly, Savant can automate a series of tedious screen reader actions on the control elements of the application when prompted by a natural language command from the user. These commands can be flexible in the sense that the user is not strictly required to specify the exact names of the control elements in the command. A user study evaluation of Savant with 11 blind participants demonstrated significant improvements in interaction efficiency and usability compared to current practices.

CCS Concepts

• **Human-centered computing** → **Accessibility technologies**; **Empirical studies in accessibility**.

Keywords

Blind users, Computer Interaction, Assistive technology, Uniform interaction, Large language models (LLMs), Accessibility

*Both authors contributed equally to this research.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ASSETS '24, October 27–30, 2024, St. John's, NL, Canada
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0677-6/24/10
https://doi.org/10.1145/3663548.3675605

ACM Reference Format:
Satwik Ram Kodandaram, Utku Uckun, Xiaojun Bi, IV Ramakrishnan, and Vikas Ashok. 2024. Enabling Uniform Computer Interaction Experience for Blind Users through Large Language Models. In *The 26th International ACM SIGACCESS Conference on Computers and Accessibility (ASSETS '24), October 27–30, 2024, St. John's, NL, Canada.* ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/3663548.3675605

1 Introduction

People who are blind depend on assistive technology, specifically a screen reader (e.g., JAWS [1], VoiceOver [2], and NVDA [3]), to interact with computer applications. The screen reader reads the content on the screen, allowing blind users to navigate and interact with different control elements and content sections of the application using keyboard shortcuts. However, modern computer applications feature complex and densely packed graphical user interfaces (GUIs) that are mainly designed for visual *point-and-click* interactions using a mouse, rather than relying on keyboard shortcuts, the principal method of interaction for blind individuals. As a result, blind people often face difficulties in performing basic tasks (such as accessing ribbon commands) in applications [24].

These interaction challenges are compounded by the heterogeneity of the applications' interfaces that require blind users to remember specific keyboard shortcuts and navigation methods for each of these applications. For example, in VLC Media Player, the keyboard shortcut to play or pause audio is "Spacebar", while in Windows Media Player, it is "Ctrl + P". Although both applications fulfill the same purpose, blind users must recall and distinguish between these unique keyboard shortcuts, increasing their cognitive burden and possibly impeding their interaction efficiency. In addition, different screen readers often require different keyboard shortcuts to execute identical tasks, resulting in differences in user interaction between platforms. For instance, in *NVDA*, the standard key combination to access a menu is usually *NVDA + N*, while in JAWS, it is *Insert + F10*. These variations in shortcuts persist when performing the same task in versions of identical applications on various platforms, e.g., in Word on Windows, the key combination

[1] https://www.freedomscientific.com/products/software/jaws/
[2] https://www.apple.com/accessibility/mac/vision/
[3] https://www.nvaccess.org/

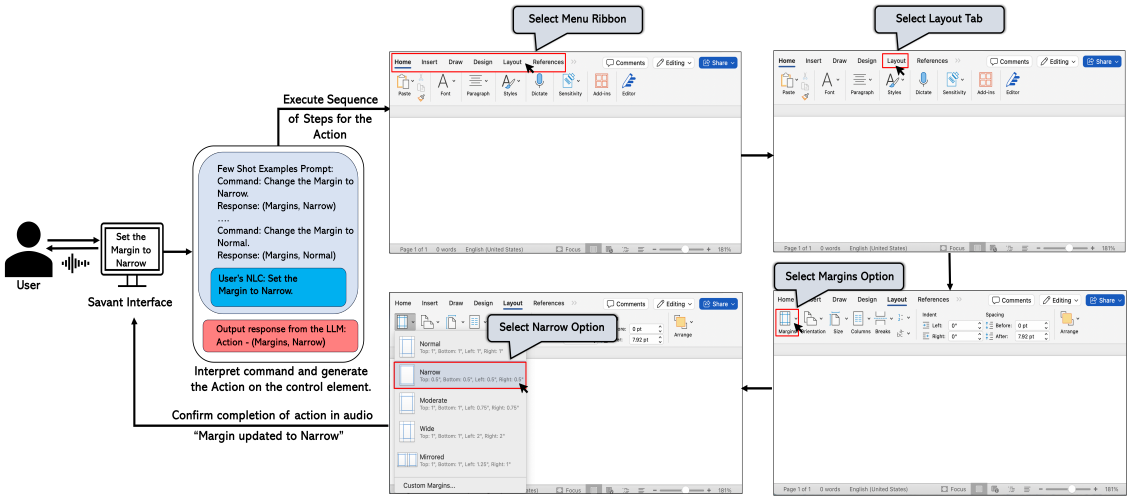


Figure 1: LLM-powered automation of sequence of screen reader steps prompted by the user's command in natural language

Alt + Ctrl + F is used to add a footnote, while in the MacOS version of Word it is Command + Option + F. These challenges have arguably hindered blind users in effectively mastering even accessible desktop applications, with research indicating that they are frequently ten times slower than sighted individuals when performing similar computer tasks [13, 26, 46].

In a previous study, blind participants emphasized the importance of having uniform and consistent access independent of the specific application, screen readers, and platforms [10], and the absence of this type of access in contemporary assistive technology was considered a significant drawback by all study participants. Assistive technology that incorporates approaches to offer this type of uniform access sought by blind users has not received much attention and is therefore the focus of our work in this paper.

We specifically introduce Savant, a novel assistive technology powered by large language models (LLMs) that augments the screen reader for enabling blind users to interact uniformly with any application interface using natural language. Uniquely, Savant simplifies and speeds up interactions with screen readers by automating a sequence of laborious screen reader actions on the application's control elements when triggered by a natural language command (NLC) from the user. These commands offer flexibility as they do not strictly require the user to identify the precise names of the control elements in the NLC.

As an illustration of Savant, consider Joe, a blind individual preparing a Word document. Suppose, for starters, that he wants to alter the standard margin "normal" to "narrow". Should Joe were to rely solely on his screen reader, his first step would be to engage the *Menu Ribbon* by hitting *ALT + H*. Following this, he would traverse the tab list using the *right arrow* key to shift focus to the *Layout* tab. Subsequently, he would explore the *Layout* choices using the *Down Arrow* key, opt for *Margins*, and finally select *Narrow*. On the other hand, with Savant, Joe simply needs to say "set the margin to narrow" to do the same task, and in response, Savant with the aid of an LLM, interprets the intent in Joe's transcribed command as an Action on a control element. It then identifies the series of screen reader steps required to implement the Action and autonomously executes this sequence of steps for Joe.

As depicted in Figure 1, Savant correctly interprets the intention of the spoken command "set the margin to narrow" as an Action on the pair <Margins, Narrow>, which amounts to assigning the value "Narrow" to the control element "Margins". It identifies the series of screen reader steps required to implement the Action <Margins, Narrow>, namely, *Select Menu Ribbon, Select Layout Tab, Select Margins, Select Narrow Option*, to implement the margin change, and finally executes them on the Word application interface in the correct sequential order. Note that in the absence of Savant, these tedious steps would have to be done manually by Joe. Significantly, even if Joe had said "change the margin to narrow", or "update the margin to 0.5", or "I want to create a narrow margin", Savant would still have accurately interpreted all of these commands given the powerful reasoning capabilities of its LLM, leading to the same series of screen reader actions on the Word application interface, underscoring the power and flexibility of Savant. Finally, Savant enables Joe to use the same command to adjust margins in other similar applications (e.g., WordPad, Pages) as well, thereby obviating the need to remember distinct shortcuts for these applications. Thus, Savant offers a promising approach to achieving consistent access across diverse interfaces.

LLMs belong to a class of foundational models that are trained on vast amounts of data, enabling them to comprehend, analyze, and produce natural language and various other forms of content for a wide variety of tasks. Their notable successes and impact on numerous fields have gained them widespread recognition. However, leveraging their capabilities in assistive technologies, as exemplified by the Savant system outlined in this paper, is a relatively unexplored topic.

Savant harnesses the capabilities of LLMs in text comprehension to offer a natural language command interface for uniform interaction across various application interfaces. The design of Savant is described in Section 3. To assess the effectiveness of Savant in practice, we conducted a user study with 11 blind participants. Savant showed significant improvements in both perceived usability (3 times higher SEQ score) and task workload (3 times lower NASA-TLX score) over status-quo screen readers, for typical computer tasks that involve accessing controls in a diverse set of applications. Participants were also able to access application controls nearly 4 times faster on average than with screen readers alone while doing these tasks. The accuracy of LLM-based interpretation and automation of participant commands was also high with a 0.78 accuracy score.

In summary, the contributions of this paper are the design, development, and evaluation of a novel assistive computer technology for blind users, namely Savant, which takes advantage of LLM to provide uniform and efficient natural language-based access to the controls of any computer application. It should be noted that Savant could also benefit the elderly or those with motor impairments who may struggle with the precision required for traditional mouse and keyboard input.

2 Related Work

2.1 Computer Usability for Blind Users

While there have been plenty of works addressing the accessibility of computer applications [14, 21, 23, 41, 42, 53], including the availability of guidelines and recommendations [19, 34], the research on usability, i.e., the ease, efficiency, and satisfaction with which blind users can do their computer tasks, is still in its infancy, with very few efforts investigating the usability of select computer applications [7, 27, 50]. As modern computer applications all have two-dimensional graphical user interfaces primarily designed for sighted interaction, there is an inherent mismatch with the one-dimensional navigation supported by a screen reader, thereby resulting in numerous usability issues as uncovered by the findings of prior studies [9, 30, 46, 49, 50]. For instance, an early work by Shinohara et al. [39] illuminated the navigational difficulties faced by blind users in computer applications. Their study notably revealed that even a single accidental key press could utterly disorient a blind user in the application UI, and moreover, recovery from such errors often entailed significant mental effort and time. In another study, Billah et al. [10] observed that differences in shortcut mappings between different screen readers and between different applications generated confusion and interaction problems for blind screen reader users. A recent work by Islam et al. [21] revealed that screen reader users' perceptions of usability and accessibility of desktop applications are nuanced, incorporating various factors beyond the traditional definitions. These factors include independence, ease of learning an application, ability to describe it to others, reliability and deterministic behavior of keyboard shortcuts, and transfer ability of knowledge to others.

Apart from the above works studying the usability issues in computer applications, there also exist research works proposing assistive-technology solutions for improving the usability of computer applications. However, these works have largely centered on the web [6, 12, 15, 16, 43] and mobile applications [11, 22, 27–29]. For instance, quite a few natural language assistants and web automation techniques have been proposed in the literature for enabling convenient web interaction for blind users [4, 5]. Assistive technologies based on third-party customized hardware have also been presented to improve navigational efficiency in web screen reading. Similarly, there have been other works that have tried to improve usability by modifying the DOM of the web pages to better suit screen-reader navigation [8].

In contrast to these web and mobile usability solutions, assistive technologies for enhancing desktop-application usability have been relatively scarce [6, 34]. Moreover, almost all these extant works have primarily targeted certain specific aspects of select computer applications. For instance, a few works in this regard [13, 25, 35, 38, 47] have specifically targeted usability of certain features of word-processing applications, such as ribbon controls, collaborative elements such as user comments, and dynamic document updates. Similarly, Doush et al. [14] created a multi-modal solution for customizing non-visual navigation and reading of tabular data in the *Microsoft Excel* application. There also have been application-specific usability solutions for PDF software [44, 45], Email application [49], and Office applications [31]. On the other hand, Uckun et al. [46] proposed a general system-wide solution for the Windows OS platform, namely *AccessBolt*, that leveraged Microsoft's UI Automation tree to provide an alternative popup interface for sequentially accessing application controls *in-one-place*, instead of navigating all over the application GUI to access these controls. While AccessBolt showed significant improvements in usability in a user study with blind participants, it also exhibited certain key limitations. First, AccessBolt could only provide access to 'visible' controls on the screen; controls not in view (e.g., controls in dropdown menus, inactive ribbons) were not accessible in AccessBolt's popup interface. Moreover, sequential navigation of controls in the popup still requires considerable time and shortcut presses, especially in complex applications such as productivity tools that have hundreds of controls. In contrast, Savant shields the user from all these low-level screen-reader operations and related GUI-navigation intricacies, thereby enabling a more convenient and uniform interaction via flexible natural language commands for blind screen-reader users.

2.2 Natural Language Assistance for Blind Users

Many commercial voice-activated assistants are now available on desktops and smartphones, such as Apple's Siri [3], Google Assistant [18], and Microsoft's Cortana [33], that enable users to automate certain computer tasks such as setting reminders and alarms, taking notes, making Facebook/Twitter updates, etc. However, these assistants provide little-to-no support for in-application activities such as accessing and invoking controls [4, 7]. For example, a user can ask Siri to open the Word application on MacOS but cannot instruct it to *set the margins*, *track changes*, etc. Even in native MacOS applications such as Finder, Siri is not able to automate simple tasks such as renaming a file

In addition to the aforementioned system-wide assistants, there also exist application-specific assistants (typically offered by the application vendor) that enable users to access features and automate tasks within the corresponding applications. For example, Google

Docs has an in-built assistant that can automate document formatting activities such as font, size, and color [4]. Recently, Amazon has enabled a natural language assistant namely Rufus [5] in their mobile application for select customers, to enable quick and easy access of product-related information via natural language queries. Similarly, many other apps are providing their own chatbots and assistants to enhance their usability [40]. The obvious drawbacks of these assistants are that they are tightly bound to their host application and moreover the underlying technologies are closed source, therefore, it is difficult if not impossible to replicate these assistants for general use across all applications.

Other than the above assistants in commercial software products, there also exist research prototypes that offer greater flexibility in terms of the range of supported commands and are also more generic in that they support multiple applications [7, 17]. For instance, Ashok et al. proposed natural language interfaces for blind screen reader users to conveniently and efficiently navigate web applications using a wide range of spoken natural language commands [4]. Specifically, these assistants first translate the user's spoken commands into the intended browsing actions and then automatically execute these actions on the user's behalf. JustSpeak [52] is another assistant service on the Android mobile platform that can also map user utterances to basic actions such as scroll, toggle switch, toggle checkbox, and activate controls. A common aspect of these aforementioned research prototypes is that they target either web or mobile applications. Phutane et al. [36] explored the potential of conversational assistants to enhance non-visual access for blind and low vision individuals via human-like conversations. To the best of out knowledge there is no existing application-agnostic, natural language-based assistive technology for desktop applications that allows for consistent and efficient interaction across various applications and platforms. Extant prototypes for web and mobile cannot be readily adapted for desktop usage due to the inherent heterogeneity in the underlying software implementations and technologies. Savant is envisioned to fill this gap.

3 Technical Design of Savant

Notations:

We use *NLC* to denote a natural language command.

We will use *CE* to denote control elements. A CE is an interface element on which the user can execute click, select, etc.

We will use *Value(CE)* to denote a value the CE can take.

We associate an *Action* with the pair <CE, Value(CE)>, meaning execution of the Action will result in a sequence of screen reader steps that will result in the assignment of Value (CE) to CE. We will use CE, control elements, and controls interchangeably.

3.1 Preliminary Data Collection

To understand the types of *NLCs* that Savant can expect to receive from blind users while they interact with computer applications, we conducted a Wizard-of-Oz (WOZ) study with 11 blind participants. In the study, the participants were asked to do an assortment of computer tasks in 5 different applications including *Excel*, *Gmail*, *File Explorer*, *Word*, and *Zoom*. The full details of the study including the design, tasks, participant details, and procedure, are provided in Appendix A.3. Overall, from the WOZ study, we collected a total 145 NLCs, and each NLC was manually annotated with the <CE, Value(CE)> pair of the corresponding application. This dataset provided the important *seed* data for both our model training and testing as explained next.

3.2 Overview

Savant system architecture consists of two components, a Preprocessing component, and a Runtime component – see Figure 2. In the figure, the upper half, shown in blue, denotes the Preprocessing component, whereas the Runtime component is shown in gray.

The objective of the Preprocessing piece is two-fold: (1) to create a non-visual representation of the application GUI that will contain information that will be used to generate the sequence of screen reader steps to invoke an action on a control element (see "Application Control Tree" in the figure); (2) to develop a dataset of few-shot samples for the application to serve as prompts (shown as "Few-Shot Examples Dataset" in the figure).

Note that preprocessing is done *only once* for each application. The application's Application Control Tree (ACT) and its Few-Shot Examples Dataset (FED) are repeatedly used at runtime to generate the sequence of screen reader steps that are required to execute the user's NLC as shown in the Runtime Component in Figure 2.

The *Crawler* module in the figure constructs the ACT by employing the UI Automation tool [32] to extract all the control elements of the application, inclusive of meta-data (e.g., control element names, control types, hierarchical relationships between control elements, actions on the control elements like clicking, selection, etc.).

The *Pair-Generator* module is responsible for the creation and storage of <CE, Value(CE)> pairs for every control element in the application, using the ACT's meta-data as a reference. To illustrate, consider Microsoft Word. Since one possible value for *Margins* is *Narrow*, the resulting pair would be *(Marin, Narrow)*. Similarly, it can take other possible values such as Normal, Moderate, Wide, etc., and a corresponding pair for each such value will be created.

The *Seed LLM* module generates the few-shot examples dataset for the application and saves it in the FED. Each example in the FED is a triple <NLC, CE, Value(CE)>. The semantics of the triple provide an interpretation for NLC, that is, linking the action of assigning Value(CE) to the control element CE.

For this purpose, we provide prompts to the *Seed LLM* to generate the triples in FED. The prompt for *Seed LLM* includes a few hand-crafted few-shot examples and a <CE, Value(CE)> pair originating from the *Pair-Generator* module. The LLM output is an NLC corresponding to the <CE, Value(CE)> pair and is stored in the FED. The hand-crafted prompts instruct the Seed LLM on the creation of the NLC for each <CE, Value(CE)> pair. It is important to note that these hand-crafted prompts are done once. They are generic and are used in the generation of the FED dataset for any application.

The objective of the Runtime component is to interpret the user's NLC, generate the correct <CE, Value(CE)> pair based on the interpretation, associate an Action on the <CE, Value(CE)> pair, and execute a sequence of screen reader steps corresponding to the Action.

[4]https://support.google.com/docs/answer/4492226?hl=en&ref_topic=1361462&sjid=13413164066125249767-NA

[5]https://www.aboutamazon.com/news/retail/amazon-rufus

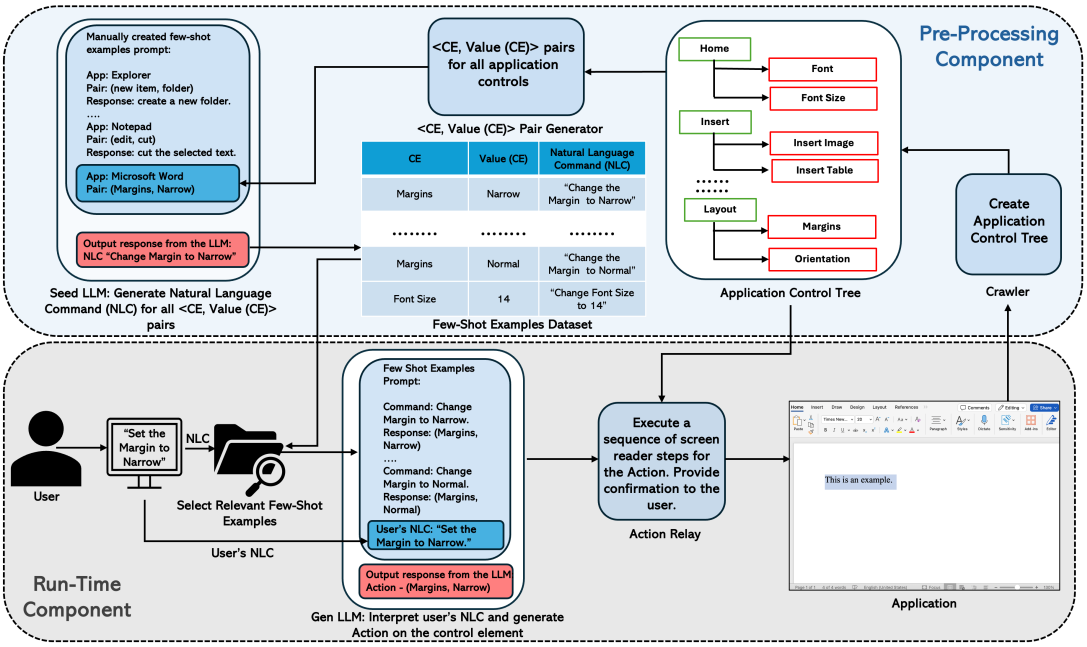


Figure 2: Architectural workflow of the Savant system. CE stands for Control Element. Value (CE) stands for the value of the Control Element. NLC stands for Natural Language Command.

Gen LLM carries out the interpretation and mapping of the NLC to the correct <CE, Value(CE)> pair. The prompt it uses comprises the user's NLC input along with a collection of few-shot examples that are pertinent to the NLC. The *Action Relay* module generates the sequence of steps in the screen reader to execute the Action on the <CE, Value(CE)> pair. To this end, it references the information in ACT that facilitates the execution of the step sequence. Details of the two components are provided in the following sections.

3.3 Preprocessing Component

Recall that the Preprocessing Component will build the ACT (the Application Control Tree) and the FED (the few-hot examples dataset). Their construction is described in the following sections.

3.3.1 ***Constructing Application Control Tree.*** We developed a custom *Crawler* module to build the ACT made up of all the control elements of the application. In the first step, the *Crawler* module utilizes the Microsoft UI Automation tool [32] to extract all visible controls of the application and other related metadata. Visible controls are those seen on the screen of the application window (e.g., Control Name, Control Type, etc.). However, controls that are not visible (e.g., sub-menus) typically do not appear in the UI Automation tree unless a user interaction prompts them. Extracting them is a bit tricky. The approach we use is as follows.

To extract these hidden controls, the *Crawler* explores visible control elements breadth-first and automatically simulates user interactions on each of these elements, such as clicking on drop-down menus or tabs, to uncover hidden controls embedded within the visible controls. For example, in Microsoft Word, when the "font family" drop-down is expanded, it reveals all the selectable font types. When the "Insert" tab is selected, it shows all the controls in that tab. By executing these simulated interactions, invisible controls are made visible in the UI Automation tree. The *Crawler* captures these newly revealed controls along with their metadata as mentioned in the overview in Section 3.2. This process is done recursively for nested controls (e.g., sub-menus). Throughout this recursive process, the *Crawler* maintains a record of the actions that reveal new controls and associates these actions with the corresponding control elements, thus generating a hierarchical tree structure that accurately reflects the desktop application's user interface hierarchy.

Within this tree structure, the nodes represent the control elements of the application, and the edges denote the actions performed on the parent node to access the child node. Take, for instance, a node that represents the "Margins" control element in Microsoft Word. The edge from its parent node, namely, the "Layout tab" control element, is labeled by the action *Click*, meaning that a click on the "Layout tab" control element reveals the"Margins" control element, among others. The"Margins" node has several outgoing edges to the children nodes, one child node per setting option. For example, under the"Margin" node, "Narrow" is a child node and the label on its edge from the parent "Margins" is labeled *Select Narrow*. The path in the tree for this selection is: (Layout->Margins->Narrow). The user actions necessary to navigate and adjust the margin settings of the document is also captured on this path as metadata.

This combination of node and edge offers a comprehensive and hierarchical depiction of the tree of control elements and actions that can be executed on them (see ACT in Figure 2).

3.3.2 *Creating the Few-Shot Examples Dataset*.

Pair generation. The *Pair Generator* module utilizes the previously built ACT to create <CE, Value(CE)> pairs for each of the application controls. For example, in *Word*, the *Editing* control element provides three choices: *Editing*, *Reviewing*, and *Viewing*. These choices are the values for the *Editing* control element, which will result in these three <CE, Value(CE)> pairs to be created: <Editing, Editing>, <Editing, Reviewing>, <Editing, Viewing>. However, some control elements might not have corresponding values; for instance, the *Bold* control element in *Word* does not provide any options. In such a case, the value for the *Bold* control element is set as *none*.

Few-shot examples generation. Recall from Section 3.2 on the overview that the *Seed LLM* module generates the few shot examples dataset for the application and saves it in the FED. Each such example in the FED is a triple <NLC, CE, Value(CE)>. The semantics of the triple provide an interpretation for NLC, namely, linking the action of assigning Value(CE) to the control element CE.

For this purpose, we provide prompts to the *Seed LLM* to generate the triples in the FED (Few-Shot Example Dataset - see Figure 2). The prompt for *Seed LLM* includes a few hand-crafted few-shot examples, some chosen from the WOZ study (see Section 3.1), and a NLC corresponding to each example as a response (see *Seed LLM* in Figure 2). These hand-crafted prompts instruct the Seed LLM on the creation of the NLC for <CE, Value(CE)> pairs. Given a <CE, Value(CE)> pair originating from the *Pair-Generator* module, the *Seed LLM* output is a NLC corresponding to the <CE, Value(CE)> pair that gets stored in the FED. We utilized the *GPT-4* model [1] with few-shot learning, as the *Seed LLM*, to produce NLCs for each <CE, Value(CE)> pair for the application.

These few-shot examples were general and varied in several aspects. First, they covered a number of diverse applications, such as file management (*Explorer*), text editing (*Wordpad*, *Notepad*, and *Microsoft Word*), video conferencing (*Zoom*), music streaming (*Spotify*), and email (*Outlook*). The diversity of applications ensured the model's ability to generalize across various computer applications and functionalities. Second, the examples included a broad range of command types, like creating new items (e.g., new folder), text formatting (e.g., strike-through), meeting management (e.g., initiating a new meeting with video off), text editing (e.g., cut), formatting adjustments (altering font size), playback controls (e.g., pause) and interface display (e.g., display the to-do bar). The breadth of command types ensured that the model was trained to generate a wide range of accurate NLCs. The prompt snippet of few-shot examples along with the output from the *Seed LLM* appears in the Appendix A.1. *It is important to note that these hand-crafted prompts are generic and that the same set of few-shot examples is used to generate the FED dataset for any application.*

Curating the FED dataset. The FED consists of few-shot examples for 11 different applications (see Appendix A.4) and for all the controls of these applications. Smaller applications, such as *Notepad*, provided roughly 50 examples, whereas larger applications, like *Excel*, offered close to 400 examples. Each example was independently assessed by two members of our research team for the accuracy of the generated few-shot examples. Examples found to be erroneous were discarded. For example, within the *Excel* application, we came across a pair <CE, Value(CE)> (coming soon, none), which the *Seed LLM* was unable to translate into a pertinent natural language command and hence was discarded. In a similar vein, in *Word*, the <CE, Value(CE)> pair <less, none> was linked erroneously to the command "Reduce the size of the selected text," After cleaning the data set, we converged on 1, 200 high-quality few-shot examples in the FED.

3.4 Run-Time Component

The runtime component will interpret the user's NLC by generating an accurate <CE, Value(CE)> pair reflecting the interpretation, associate an Action on the <CE, Value(CE)> pair, and execute the sequence of steps corresponding to the Action.

The *Gen LLM* module is responsible for producing a <CE, Value(CE)> pair that mirrors the interpretation of the user's NLC. This process leverages the capabilities of GPT-4 with few-shot learning. We describe this process next.

The prompt it uses comprises the user's NLC input along with a collection of few-shot examples that are pertinent to the NLC. The *Action Relay* module generates the sequence of steps in the screen reader to execute the Action on the <CE, Value(CE)> pair. To this end, it references the information in ACT that facilitates execution of the step sequence.

3.4.1 ***Prompt Engineering***.

The prompt it uses comprises the user's NLC input along with a collection of few-shot examples that are relevant for user's input NLC. Relevancy is calculated using the *BERT* module. Using BERT word embedding, it computes the semantic similarity of user's NLC and the few-shot example data set of the application in the FED. The result of the *BERT* module is a set of relevant few shot examples for the user's NLC. For example, if the user command is "Set the Margin to Narrow", the semantic similarity search based on BERT embedding will pull all related examples linked with the *Margin* control element such as "Change the Margin to Narrow, (Margins, Narrow)", "Change Margin to Normal, (Margins, Normal)", "Change Margin to Moderate", etc (see *Gen LLM* part in Figure 2). Using word embedding to retrieve relevant few-shot examples is crucial to ensure that *Gen LLM* generates a <CE, Value(CE)> pair that is an accurate reflection of the interpretation of user's NLC.

User's NLC and the relevant few shot examples are used as prompts for *Gen LLM* to generate a <CE, Value(CE)> pair demoting the interpretation of the NLC. A prompt containing several few shot examples and the output for a sample user command is included in the appendix A.2.

3.4.2 ***Action Relay***.

The *Action Relay* module generates the sequence of screen reader steps to execute the Action on the <CE, Value(CE)> pair generated by *Gen LLM*. To this end, it references the information in ACT that facilitates the execution of the step sequence. For example, when the NLC 'Change font size to 12' is processed, the *Action Relay* module carries out the screen reader steps in the following order: *Select Font Group Menu*, *Select Font Dropdown*, and *Select Font Size to 12*. Once the task is accomplished, the system provides a 'Font size updated 12' confirmation message to the user. If the action fails, the system prompts the user to re-issue the command and the process begins anew.

ID	Gender	Age	Acuity	Screen Reader
P1	Male	43	Light Perception	JAWS, NVDA
P2	Male	58	No Vision	JAWS, NVDA
P3	Male	40	Light Perception	JAWS, NVDA, VoiceOver
P4	Male	36	No Vision	JAWS
P5	Female	39	No Vision	JAWS, NVDA, VoiceOver
P6	Female	64	Light Perception	JAWS
P7	Female	61	No Vision	JAWS
P8	Female	31	Light Perception	JAWS, VoiceOver
P9	Female	35	No Vision	JAWS, NVDA, VoiceOver
P10	Male	44	Light Perception	JAWS, NVDA, VoiceOver
P11	Female	35	Light Perception	JAWS, VoiceOver

Table 1: User study participant demographics.

3.4.3 ***Handling Ambiguity.*** On occasions, a user's NLC may be large. For instance, the *Notepad* application: If a user instructs "erase the highlighted text", they may be referring to either the *delete* or *cut* function. The Gen LLM will need to present both of these options to the user for disambiguation. Fortunately, the underlying *GPT-4* driving *Gen LLM* can be configured to provide multiple responses in rank order. A simple postprocessing step Savant will present these choices to the user only when the responses differ.

3.4.4 ***Evaluation of Gen LLM.*** We used the earlier ground-truth WOZ study dataset to evaluate the performance of Gen LLM. For each example in the WOZ dataset, we fed the *NLC* as input to our model to generate the output of the <CE, Value(CE)> pair, which was then compared to the ground truth pair in the data set. The overall accuracy of our Gen LLM was 78% (113 out of 145 *NLCs*).

We also analyzed the pattern where the model gave incorrect outputs. We noticed that incorrect outputs from *Seed LLM* were primarily caused by natural language commands containing multiple instructions within a single command. For example, some of the tasks in our WOZ study were multi-step tasks, meaning they can be completed by giving a single *composite* command or multiple single-action commands. For example, to move the dog picture to the photos folder, a participant can use the *move* composite query or use *Copy* and *Paste* single action commands. However, many participants opted to include both *Copy* and *Paste* within a single command, which *Gen LLM* was unable to process and generate the action. This limitation is discussed further in the Section 5.

4 User Study

We conducted an IRB-approved user study with blind screen reader users to test the effectiveness of Savant against the status quo.

4.1 Participants

We recruited 11 blind screen reader users to participate in the study. Recruitment was done by leveraging an email contact list followed by snowball sampling. The inclusion criteria were: (i) having a visual impairment severe enough to require screen readers for interacting with computers; (ii) proficiency with *JAWS* screen reader software; and (iii) experience with the *Windows* operating system and software applications. The average age of the participants was 44.2 (median=40, min=35, max=64), and the gender representation was almost equal (5 male, 6 female). All participants reported owning a desktop or laptop computer. Additionally, they were all very comfortable using the *JAWS* screen reader software, with 6 participants also having experience in using *NVDA* and 6 using *VoiceOver*. More self-reported details of the participant demographics can be found in Table 1. There was no overlap between the participant pools of the earlier WOZ study and this study, thereby promoting external validity.

4.2 Study Design

In the study, participants had to complete typical computer tasks in 6 different applications: *Amazon* (opened in *Firefox*), *Excel*, *File Explorer*, *Google Docs* (opened in *Chrome*), *Outlook*, and *Spotify*. Each of these applications represented a unique use case (e-commerce, productivity, text editing, e-mailing, etc.) to evaluate the generalizability of our system. Exact details of the tasks assigned for each application are shown in Table 2. We selected these tasks as representative and useful for screen reader users.

The study had a within-subject design where participants interacted with an application under 3 conditions. These conditions were:

- **Screen Reader (JAWS) only:** Participants could only use JAWS screen reader software.
- **Screen Reader with a state-of-the-art AccessBolt interface:** Participants were allowed to use a state-of-the-art AccessBolt system [46] that (as explained in Related Work) enabled screen reader users to access all the *visible* controls via a custom pop-up interface.

- **Screen Reader with Savant's command interface:** Participants were allowed to use Savant's natural language interface to do the assigned tasks.

Note: AccessBolt and Savant differ in their interface and accessibility approaches. AccessBolt utilizes a pop-up interface that displays all visible control elements, requiring users to manually navigate and select these controls to perform tasks. Consequently, interaction is confined to visible control elements only, requiring users to manually reveal invisible controls for AccessBolt to access them. In contrast, Savant offers a natural language interface, allowing users to execute tasks through natural language commands (NLC), providing a more intuitive and seamless interaction experience that doesn't restrict users to visible controls only.

Participants interacted with 2 different applications under each condition. The ordering of the conditions and applications across participants was counterbalanced using the Latin square method [6].

4.3 Apparatus

The study was conducted in person, where the participants were provided with a Lenovo ThinkPad Windows laptop, which had all the necessary applications installed, including the *JAWS* screen reader. An externally accessible QWERTY desktop keyboard was also connected to the laptop, as all participants mentioned that they were familiar with the standard keyboard during the recruitment process.

4.4 Procedure

The experimenter first explained the purpose of the study to the participants and obtained their informed consent. Then, the participants were given 20 minutes to practice/revise their *JAWS* skills, configure JAWS to best match their preferences, and get comfortable using the AccessBolt and Savant assistive technologies. For the practice session, we used the *WordPad* application as the context, where the experimenter showcased the range and flexibility of command usage in Savant to the participant. After the practice session, the participants we asked to complete the tasks under different conditions in the predetermined counterbalanced order. If the participant could not finish a task within 10 minutes, it was considered a failure. After each task, an SEQ [7] question was asked, and after completing all the tasks, the NASA-TLX questionnaire [20] was administered. Lastly, the experimenter concluded the study with an open-ended exit interview where the participant was asked to provide subjective feedback, including feature requests and suggestions for improvement. With the participant's permission, recording features were active throughout the study to capture all interaction activities for post-study analyses. Each study lasted for nearly 2 hours. All conversations were in English, and the participants were compensated monetarily for their time and contribution.

4.5 Measurements

From the study, we collected the following metrics: (i) the average time spent on each task; (ii) the average number of shortcuts pressed per task, including navigation; (iii) the scores from the Single-Ease-Questionnaire (SEQ) on a 7-point Likert Scale, where 1 represents 'Hard' and 7 'Easy'; and (iv) NASA-TLX questionnaire responses for each condition. The SEQ was used to assess the perceived difficulty of each application, while the NASA-TLX questionnaire was administered after each condition to gauge the participants' perceived workload. We used qualitative analysis methods for the textual data collected by transcribing the participants' utterances and qualitative feedback from the exit interviews. Specifically, we used the standard open coding and axial coding techniques [37] to iteratively annotate the textual data and excavate key insights and themes recurring in the data. We detail our findings next.

4.6 Results

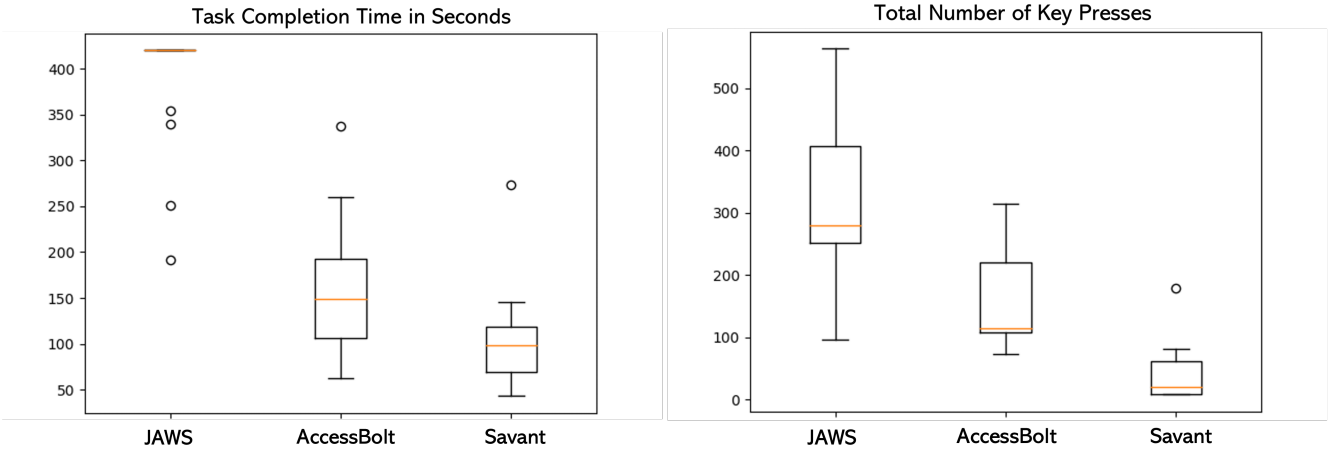
4.6.1 ***Completion Rate.*** The completion rate varied across study conditions. In the JAWS condition, the participants completed only 6 out of the 22 tasks. The completion rate in the AccessBolt condition was much higher (14 out of the 22 tasks), however it was the highest in the Savant condition (18 out of 22 tasks). A closer analysis of the user behavior during the tasks revealed that in the JAWS condition, one of the main reasons the participants struggled to complete the tasks was because of *invisible* controls that could be accessed only by opening up the corresponding 'container' control. For instance, participants had difficulty locating the *Date* sorting option in *Outlook*. This option was nested within the *Sort By* sub-menu, found under the *View Menu*. Specifically, in 8 of the 16 completion failures in the JAWS condition, the participants tried to locate the control by sequentially navigating over the visible GUI elements on the screen without realizing that they had to first select a parent control (e.g., View Menu) before they could access the desired target control (e.g., Sort By). In these 8 cases, only after trying for some time and failing to locate the target control the participants explore other strategies, such as exploring menus, sub-menus, and tabs, but they ran out of time before they could find the target control. A similar pattern was observed in the AccessBolt condition, where in 6 out of 8 completion failures, the participants unsuccessfully tried to locate the target control in its pop-up interface, and eventually ran out time when they were trying to find the control in the application's GUI. The few failures in the Savant condition were all due to the LLM errors where user commands could not be accurately mapped to the desired target control, despite multiple attempts by the participants.

4.6.2 ***Average time spent per task.*** On average, the participants spent 395 seconds under JAWS condition (median=420, SD=61.1), 159 seconds under AccessBolt condition (median=148.5, SD=69.6), and 101 seconds under Savant condition (median=98, SD=49.5) to do the tasks. Figure 3a shows the box plot for the task completion times for the three conditions. A Shapiro-Wilk test showed that the data for JAWS (W=0.478, p=$8.15e^{-8}$) and Savant deviated from normal distribution (W=0.821, p=0.001) whereas AccessBolt data had normal distribution (W=0.944, p=0.236). First, we conducted a Wilcoxon Signed Rank test between JAWS and AccessBolt, which showed that the difference between the completion times was statistically significant (W=0.0, p=$4.0e^{-5}$). Second, we ran the same test between AccessBolt and Savant conditions, and the result showed statistical significance between these two conditions as well (W=51.5, p=0.01).

[6] http://compneurosci.com/wiki/images/9/98/Latin_square_Method.pdf
[7] https://measuringu.com/seq10/

Application	Task 1 Description	Task 1 Depth	Task 2 Description	Task 2 Depth
Amazon	Go to the Best Sellers tab	2	Change the language of the website	2
Excel	Insert a pie chart	3	Add a new note	2
File Explorer	Open the PDF document with Adobe Acrobat	2	Move the dog.png to the Pictures folder	3
Google Docs	Change font size to 14	1	Insert an image	3
Outlook	Flag this email	1	Arrange emails by date	2
Spotify	Create a new playlist	2	Start a private listening session	2

Table 2: Tasks the participants had to complete in the user study. Depth represents the number of user interactions needed to navigate from the root to a specific element to complete the task.



(a) Box plot showing task completion times in seconds.

(b) Box plot showing the total key presses recorded under each condition.

Figure 3: Box plots for quantitative measurements collected during the user study.

As explained previously, in the JAWS and AccessBolt conditions, the participants spent considerable time searching for the target control in the application's GUI, which was especially the case for 'invisible' controls. The longest duration a participant persisted was 7 minutes before giving up. In contrast, the Savant condition eliminated this search process; participants could simply issue a natural language command, and access to the control was automated. Most of the time spent in the Savant condition was allocated to processing the natural language command by the LLM, with a portion dedicated to executing the entire pipeline. In some cases, the participants also had to rephrase or repeat the command because the LLM was unable to correctly attribute the command to the corresponding application control (due to speech-recognition errors as well as ambiguity in utterances). Occasionally, due to ambiguity in the given natural language command, the system presented multiple options to the user. In these instances, participants spent additional time determining the correct control element.

4.6.3 ***Key Press Statistics.*** On average, the participants pressed 319 keys under JAWS condition (median=279, SD=132.8), 163 keys under AccessBolt condition (median=115, SD=189.1) and 44 under Savant condition (median=20, SD=51.9). Shapiro-Wilk test showed divergence from a normal distribution for AccessBolt (W=0.83, p=0.25) and Savant conditions (W=0.75, p=0.001). Figure 3b shows the boxplot for the total number of keys pressed. A Wilcoxon Signed Rank test between JAWS and AccessBolt conditions showed the reduction in key presses is statistically significant (W=2.0, p=0.006). Another Wilcoxon test between JAWS and Savant conditions also showed that reducing key presses was statistically significant (W=0.0, p=0.003).

Furthermore, key press data were analyzed to determine the proportions of basic navigation keys (Tab and Arrow keys) used by the participants within the total key presses during the tasks. The analysis revealed that in the JAWS condition, a striking 91% of the total key presses were these basic navigation keys. In contrast,

this percentage decreased to 70% in the AccessBolt condition and dropped even further to 30% in the Savant condition. This shift in key press distribution underscores the differences in interaction behavior across the three conditions. In the JAWS and AccessBolt conditions, participants predominantly relied on navigation keys to search for the target control either within the application GUI or the AccessBolt's popup interface. On the other hand, in the Savant condition, the use of navigation keys was naturally lesser due to the availability of natural language input alternatives.

4.6.4 ***NLC Analysis.*** Out of 22 tasks assigned to the participants in the Savant condition, 11 were successfully completed on the first attempt (i.e., the LLM correctly identified the <CE, Value(CE)> pair), while 7 were completed after multiple tries. This repeated interaction contributed to longer completion times. 4 tasks remained incomplete despite repeated attempts by the participants. Upon analyzing the cases where the system produced incorrect responses, two reasons were identified. Firstly, some user commands were complex (e.g., "Select this cell and add a note"), consisting of multiple instructions within a single command. These commands often required several steps and involved accessing multiple controls. Our system is presently not equipped to handle a complex command and access multiple controls in one shot, leading to errors in execution. Secondly, errors in automatic speech-to-text conversion occasionally occurred, preventing the system from accurately interpreting the command and necessitating the user to repeat the command.

4.6.5 ***Single Ease Question Score.*** We used the standard Single-Ease Question (SEQ) score to assess usability and to determine the perceived difficulty of tasks. The SEQ questionnaire asks participants to rate on a scale of 1 to 7, with 1 being hard and 7 being easy. On average, the tasks got a rating of 2 under JAWS condition (median=2, SD=1.19), 5.6 under AccessBolt (median=6, SD=1.61) and 6.57 under Savant (median=7, SD=0.80). Shapiro-Wilk tests showed a departure from normality for all the conditions. A Wilcoxon test between JAWS and AccessBolt showed that the improvements in user experience were statistically significant (W=2.0, p=$7.36e^{-5}$). Another Wilcoxon test between AccessBolt and Savant also showed statistical significance (W=15.0, p=0.01).

4.6.6 ***NASA-TLX Score.*** We administered the standard NASA-TLX questionnaire [20] to gauge the workload experienced by the participants while they did the assigned tasks in different study conditions. NASA-TLX generates a score between 0 and 100 to estimate perceived task workload, where lower TLX values indicate lesser workloads and, hence, better performance. Overall, we observed that the study conditions had a significant impact on NASA-TLX scores. The TLX score for Savant(mean=16.16, median=15.0, SD=8.57)) condition was significantly lower than for AccessBolt (mean=28.93, median=29.16, SD=5.09) and JAWS (mean=49.9, median=45.8, SD=20.9) conditions. A Wilcoxon signed-rank test between JAWS and AccessBolt conditions showed that the difference in TLX scores was statistically significant (W=2, p=.005). Another Wilcoxon test between AccessBolt and Savant also showed statistical significance (W=2.5, p=.006). Friedman's test for NASA-TLX produced a chi-square statistic of 18.18 (df = 2) and a p-value of 0.00011, implying statistical significance at $p < .05$.

A closer inspection of the responses to individual subscales of the TLX questionnaire revealed that the Effort and Frustration subscales contributed more toward the relatively higher workload scores in the JAWS condition, i.e., the ratings for these subscales were significantly higher than those for the other subscales (Temporal Demand, Physical Demand, Mental Demand, and Overall Performance). For the AccessBolt condition, Effort was the discriminating subscale that received much higher ratings than the other subscales. The ratings for Savant, however, were uniform across all the six subscales.

4.6.7 ***Qualitative Feedback.*** A few notable observations that were uncovered from the exit interviews are listed next.

Participant preferences and feedback on Savant. All participants expressed their preference for Savant when asked about their preferred system. Participants *P*1, *P*2, *P*3, *P*6, and *P*11 praised Savant for its high user-friendliness. Participant *P*2, for instance, said, "I like the voice (Savant) because sometimes I feel lazy, and using the voice is direct and easy." Meanwhile, Participants *P*4, *P*7, *P*8, and *P*9 highlighted that they appreciated having fewer keyboard presses and simpler navigation to find the controls. According to *P*8, "Less navigation, fewer keyboard presses make everything faster." Participants *P*6 and *P*9 also pointed out that they liked how Savant had the same command interface for all applications, which they thought was a cool idea. *P*6 said, "People (screen reader users) are used to Tab and Shift+Tab controls. Performing a task with only a few familiar keys is much better." P5 stated that our system should have an alternate pop-up interface like AccessBolt Their reason was: "It (keyboard interface) gives me more opportunities when I can't use my voice: in a silent place (library), in a Zoom meeting, when there are background noises".

Suggestions for improvement. Two participants, *P*1 and *P*3, suggested that Savant could be improved by supporting more complex commands such as composite commands. To quote *P*1, "It (Savant) should support commands like open the document and copy the text". *P*5 and *P*9 suggested allowing users to change the keyboard shortcut that triggers the system or activate the command interface. Lastly, *P*1 and *P*4 suggested that our system should have a wake word for triggering instead of a push-to-talk button.

Tedious to work with keyboard shortcuts. When asked whether they had any problems with keyboard shortcuts used in computer applications, participants *P*3, *P*8, *P*9, and *P*10 complained about the heterogeneity of keyboard shortcuts used across computer applications and operating systems; *P*10 said, "Navigating a web page uses certain commands. When in an application, there are other generic controls and application-specific controls. There should be a universal guide for app developers", while *P*9 mentioned, "Sometimes shortcuts change depending on what you are doing. You need to be a power user to know them all". Some of the frustrations participants sounded were: *P*4: "I can't memorize (shortcuts), I have to go look it up in forums or ask email groups", *P*5:"You are busy with content. Then, you have to format the content, and it gets you stressed. You lose focus. Get distracted from the task". One point all participants agreed upon was that they had to keep practicing keyboard shortcuts or that they had forgotten how to do specific tasks.

5 Discussion

The results of our user study are indicative of the potential of Savant to tackle the interaction challenges faced by blind computer users, arising from the diversity of the interfaces of applications.

5.1 Limitations and Future Work

Here, we discuss the limitations of Savant and a roadmap for handling them.

5.1.1 ***Supporting Pop-up and Sub-windows Controls.*** In the qualitative feedback, numerous participants pointed out the advantages of enabling Savant to manage tasks involving pop-ups and sub-windows. To enhance our system with these features, several challenges must be tackled. First, such windows manifest unpredictably, complicating the prediction of their emergence and the controls they include. Second, a comprehensive analysis of the relationships between various pop-ups/sub-windows and the primary window is essential for extracting all application controls. Third, the complexity of the extraction process increases when multiple actions on control elements trigger the appearance of identical pop-up windows. Additionally, the controls within these pop-ups and sub-windows often show context-dependent behaviors, which poses significant difficulties in their programmatic extraction and interaction. For instance, the options on the toolbar can vary depending on whether the user is editing text, adding a table, or adjusting image formats. Addressing these issues will be the focus of future development.

5.1.2 ***Supporting Complex Commands.*** Currently, Savant is capable of executing simple single-action commands such as button clicks or selections from a drop-down menu. Feedback from participants in our study suggests that Savant should be enhanced to handle more complex commands. For instance, the command 'Make all text italic' would need to be separated into two simpler actions: *Click Select All button* and *Click italic button.* To address such complex commands, a task decomposition model similar to the one in [51], could break down complex commands into single-action tasks for Savant to execute.

5.1.3 ***Supporting Complex Task Involving Multiple Applications.*** Many computer tasks in practical situations involve multiple control elements (e.g., *load the (named) file from the Documents folder into the editor*), possibly across multiple applications (e.g., *Microsoft Word* and *Explorer*). Automating such tasks will require reasoning over potentially multiple applications' contexts, identifying application-specific intent underlying the request within each application, and seamlessly stitching together numerous user-interface operations in appropriate execution sequences. It would be possible to use LLMs for this task, a topic for future research.

5.1.4 ***Applicability to complex GUIs and sequential tasks.*** Extending our LLM-based method to complex sequential tasks is possible using a Chain-of-Thought (COT) prompting strategy [48]. This strategy helps LLMs handle complex tasks by breaking them into logical steps. For a user's natural language command C, the prompt P will include chain-of-thought in-context few-shot examples guiding the LLM in generating an accurate action sequence.

5.1.5 ***Multilingual support and future extensions.*** Large language models (LLMs) demonstrate remarkable capabilities in understanding and translating instructions from non-English languages into English. This proficiency suggests a promising avenue for enhancing Savant by integrating translation functionalities, enabling support for multiple languages. Such an expansion could significantly broaden the accessibility and usability of Savant for non-English speaking users, presenting an exciting direction for future research.

5.1.6 ***Wake-word Implementation.*** The results of the user study indicated a preference for activating the system with a *wake-word* instead of using a push-to-talk button. Participants noted that a system not dependent on keyboard input could greatly assist people unable to use keyboards due to various disabilities (such as amputation, paralysis, or motor impairments). Implementing this feature requires a continuously active microphone and a wake-word. Deploying wake-word detection in desktop applications involves several challenges, such as the need for accurate detection, instant processing, addressing privacy issues, and ensuring multi-platform compatibility. Overcoming these challenges requires thorough model training, optimization techniques, and effective management of privacy concerns.

6 Conclusion

In this paper, we addressed the usability issues faced by blind screen reader users with desktop applications. From prior studies, it was evident that blind users face several issues while interacting with computer applications, such as difficulty in memorizing keyboard shortcuts, accessing invisible controls, etc. We proposed Savant interface that can automatically access and invoke all the application controls on the user's behalf by leveraging large language models (LLM), thereby allowing users to perform tasks uniformly across arbitrary desktop applications. By employing a design process that incorporated user feedback, we have successfully showcased notable improvements in the efficiency of computer interactions and the usability of desktop applications. This progress signifies a favorable advancement towards creating a digital environment that is more inclusive for individuals with visual impairments.

Acknowledgments

We thank the anonymous reviewers for their insightful feedback. This work was supported by the Google Inclusion Research Award, NSF Awards: 2153056, 2125147, 2113485, and the following NIH Awards: R01EY030085, R01HD097188, and R01EY035688.

References

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).

[2] Amazon. 2023. Amazon Alexa Voice Assistant. https://www.amazon.com/b?ie=UTF8&node=21576558011.

[3] Apple. 2023. Siri Voice Assistant. https://www.apple.com/siri/.

[4] Vikas Ashok, Yevgen Borodin, Yury Puzis, and IV Ramakrishnan. 2015. Capti-speak: a speech-enabled web screen reader. In *Proceedings of the 12th International Web for All Conference.* 1–10.

[5] Vikas Ashok, Yury Puzis, Yevgen Borodin, and IV Ramakrishnan. 2017. Web screen reading automation assistance using semantic abstraction. In *Proceedings of the 22nd International Conference on Intelligent User Interfaces.* 407–418.

[6] Vikas Ashok, Mohan Sunkara, and Satwik Ram. 2023. Assistive Technologies for People with Visual Impairments. (2023).

[7] Vikas Ganjigunte Ashok. 2018. *Non-Visual Web Browsing: From Accessibility with Screen Readers to Usability with Assistants.* Ph. D. Dissertation. State University of New York at Stony Brook.

[8] Ali Selman Aydin, Shirin Feiz, Vikas Ashok, and IV Ramakrishnan. 2020. Sail: Saliency-driven injection of aria landmarks. In *Proceedings of the 25th international conference on intelligent user interfaces*. 111–115.

[9] Mark S Baldwin, Gillian R Hayes, Oliver L Haimson, Jennifer Mankoff, and Scott E Hudson. 2017. The tangible desktop: a multimodal approach to nonvisual computing. *ACM Transactions on Accessible Computing (TACCESS)* 10, 3 (2017), 1–28.

[10] Syed Masum Billah, Vikas Ashok, Donald E Porter, and IV Ramakrishnan. 2017. Ubiquitous accessibility for people with visual impairments: Are we there yet?. In *Proceedings of the 2017 chi conference on human factors in computing systems*. 5862–5868.

[11] Syed Masum Billah, Yu-Jung Ko, Vikas Ashok, Xiaojun Bi, and IV Ramakrishnan. 2019. Accessible gesture typing for non-visual text entry on smartphones. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. 1–12.

[12] Yevgen Borodin, Jeffrey P Bigham, Glenn Dausch, and IV Ramakrishnan. 2010. More than meets the eye: a survey of screen-reader browsing strategies. In *Proceedings of the 2010 International Cross Disciplinary Conference on Web Accessibility (W4A)*. 1–10.

[13] Maria Claudia Buzzi, Marina Buzzi, Barbara Leporini, and Giulio Mori. 2012. Designing e-learning collaborative tools for blind people. *E-Learning-Long-Distance and Lifelong Perspectives* (2012), 125–144.

[14] Iyad Abu Doush and Enrico Pontelli. 2013. Non-visual navigation of spreadsheets: Enhancing accessibility of Microsoft Excel™. *Universal access in the information society* 12 (2013), 143–159.

[15] Javedul Ferdous, Hae-Na Lee, Sampath Jayarathna, and Vikas Ashok. 2022. InSupport: Proxy interface for enabling efficient non-visual interaction with web data records. In *27th International Conference on Intelligent User Interfaces*. 49–62.

[16] Javedul Ferdous, Hae-Na Lee, Sampath Jayarathna, and Vikas Ashok. 2023. Enabling Efficient Web Data-Record Interaction for People with Visual Impairments via Proxy Interfaces. *ACM Transactions on Interactive Intelligent Systems* 13, 3 (2023), 1–27.

[17] Prathik Gadde and Davide Bolchini. 2014. From screen reading to aural glancing: towards instant access to key page sections. In *Proceedings of the 16th international ACM SIGACCESS conference on Computers & accessibility*. 67–74.

[18] Google. 2023. Google Voice Assistant. https://assistant.google.com/.

[19] Simon Harper and Alex Q Chen. 2012. Web accessibility guidelines: A lesson from the evolving Web. *World Wide Web* 15 (2012), 61–88.

[20] Sandra G Hart and Lowell E Staveland. 1988. Development of NASA-TLX (Task Load Index): Results of empirical and theoretical research. In *Advances in psychology*. Vol. 52. Elsevier, 139–183.

[21] Md Touhidul Islam, Donald E Porter, and Syed Masum Billah. 2023. A Probabilistic Model and Metrics for Estimating Perceived Accessibility of Desktop Applications in Keystroke-Based Non-Visual Interactions. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. 1–20.

[22] Yu-Jung Ko, Aini Putkonen, Ali Selman Aydin, Shirin Feiz, Yuheng Wang, Vikas Ashok, IV Ramakrishnan, Antti Oulasvirta, and Xiaojun Bi. 2021. Modeling Gliding-based Target Selection for Blind Touchscreen Users. In *Proceedings of the 23rd International Conference on Mobile Human-Computer Interaction*. 1–14.

[23] Satwik Ram Kodandaram, Mohan Sunkara, Sampath Jayarathna, and Vikas Ashok. 2023. Detecting Deceptive Dark-Pattern Web Advertisements for Blind Screen-Reader Users. *Journal of Imaging* 9, 11 (2023), 239.

[24] Hae-Na Lee, Vikas Ashok, and IV Ramakrishnan. 2020. Repurposing visual input modalities for blind users: a case study of word processors. In *2020 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. IEEE, 2714–2721.

[25] Hae-Na Lee, Vikas Ashok, and IV Ramakrishnan. 2020. Rotate-and-Press: A Non-visual Alternative to Point-and-Click?. In *International Conference on Human-Computer Interaction*. Springer, 291–305.

[26] H. N. Lee, V. Ashok, and I. V. Ramakrishnan. 2020. Repurposing Visual Input Modalities for Blind Users: A Case Study of Word Processors. In *2020 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. 2714–2721. https://doi.org/10.1109/SMC42975.2020.9283015

[27] Barbara Leporini, Maria Claudia Buzzi, and Marina Buzzi. 2012. Interacting with mobile devices via VoiceOver: usability and accessibility issues. In *Proceedings of the 24th Australian computer-human interaction conference*. 339–348.

[28] Zhi Li, Yu-Jung Ko, Aini Putkonen, Shirin Feiz, Vikas Ashok, IV Ramakrishnan, Antti Oulasvirta, and Xiaojun Bi. 2023. Modeling touch-based menu selection performance of blind users via reinforcement learning. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. 1–18.

[29] Zhi Li, Maozheng Zhao, Dibyendu Das, Hang Zhao, Yan Ma, Wanyu Liu, Michel Beaudouin-Lafon, Fusheng Wang, Iv Ramakrishnan, and Xiaojun Bi. 2022. Select or Suggest? Reinforcement Learning-based Method for High-Accuracy Target Selection on Touchscreens. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*. 1–15.

[30] Mei Miao, Hoai Anh Pham, Jens Friebe, and Gerhard Weber. 2016. Contrasting usability evaluation methods with blind users. *Universal access in the Information Society* 15 (2016), 63–76.

[31] Microsoft. 2021. Microsoft Office guide for people who are blind or low-vision. https://docs.microsoft.com/en-us/windows/win32/winauto/entry-uiauto-win32.

[32] Microsoft. 2021. Microsoft UI Automation. https://docs.microsoft.com/en-us/windows/win32/winauto/entry-uiauto-win32.

[33] Microsoft. 2023. Cortana Voice Assistant. https://www.microsoft.com/en-us/cortana.

[34] Lourdes Morales, Sonia M Arteaga, and Sri Kurniawan. 2013. Design guidelines of a tool to help blind authors independently format their word documents. In *CHI'13 Extended Abstracts on Human Factors in Computing Systems*. 31–36.

[35] Giulio Mori, Maria Claudia Buzzi, Marina Buzzi, Barbara Leporini, and Victor MR Penichet. 2011. Making "Google Docs" user interface more accessible for blind people. In *Advances in New Technologies, Interactive Interfaces, and Communicability: First International Conference, ADNTIIC 2010, Huerta Grande, Argentina, October 20-22, 2010, Revised Selected Papers 1*. Springer, 20–29.

[36] Mahika Phutane, Crescentia Jung, Niu Chen, and Shiri Azenkot. 2023. Speaking with My Screen Reader: Using Audio Fictions to Explore Conversational Access to Interfaces. In *Proceedings of the 25th International ACM SIGACCESS Conference on Computers and Accessibility*. 1–18.

[37] Johnny Saldaña. 2021. The coding manual for qualitative researchers. (2021).

[38] John Gerard Schoeberlein and Yuanqiong Wang. 2014. Usability Evaluation of an Accessible Collaborative Writing Prototype for Blind Users. *Journal of Usability Studies* 10, 1 (2014).

[39] Kristen Shinohara and Josh Tenenberg. 2007. Observing Sara: a case study of a blind person's interactions with technology. In *Proceedings of the 9th international ACM SIGACCESS conference on Computers and accessibility*. 171–178.

[40] Justina Sidlauskiene, Yannick Joye, and Vilte Auruskeviciene. 2023. AI-based chatbots in conversational commerce and their effects on product and price perceptions. *Electronic Markets* 33, 1 (2023), 24.

[41] Reeta Singh. 2012. Blind handicapped vs. technology: How do blind people use computers. *International Journal of Scientific & Engineering Research* 3, 4 (2012), 1–7.

[42] Mohan Sunkara, Sandeep Kalari, Sampath Jayarathna, and Vikas Ashok. 2023. Assessing the Accessibility of Web Archives. In *2023 ACM/IEEE Joint Conference on Digital Libraries (JCDL)*. IEEE, 253–255.

[43] Mohan Sunkara, Yash Prakash, Hae-Na Lee, Sampath Jayarathna, and Vikas Ashok. 2023. Enabling Customization of Discussion Forums for Blind Users. *Proceedings of the ACM on Human-Computer Interaction* 7, EICS (2023), 1–20.

[44] Utku Uckun, Ali Selman Aydin, Vikas Ashok, and IV Ramakrishnan. 2020. Breaking the accessibility barrier in non-visual interaction with pdf forms. *Proceedings of the ACM on Human-computer Interaction* 4, EICS (2020), 1–16.

[45] Utku Uckun, Ali Selman Aydin, Vikas Ashok, and IV Ramakrishnan. 2020. Ontology-Driven Transformations for PDF Form Accessibility. In *Proceedings of the 22nd International ACM SIGACCESS Conference on Computers and Accessibility*. 1–3.

[46] Utku Uckun, Rohan Tumkur Suresh, Md Javedul Ferdous, Xiaojun Bi, I.V. Ramakrishnan, and Vikas Ashok. 2022. Taming User-Interface Heterogeneity with Uniform Overlays for Blind Users. In *Proceedings of the 30th ACM Conference on User Modeling, Adaptation and Personalization (UMAP '22)*. Association for Computing Machinery, New York, NY, USA, 212–222. https://doi.org/10.1145/3503252.3531317

[47] Mirza Muhammad Waqar, Muhammad Aslam, and Muhammad Farhan. 2019. An intelligent and interactive interface to support symmetrical collaborative educational writing among visually impaired and sighted users. *Symmetry* 11, 2 (2019), 238.

[48] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems* 35 (2022), 24824–24837.

[49] Brian Wentz, Harry Hochheiser, and Jonathan Lazar. 2013. A survey of blind users on the usability of email applications. *Universal access in the information society* 12 (2013), 327–336.

[50] Brian Wentz and Jonathan Lazar. 2011. Usability evaluation of email applications by blind users. *Journal of Usability Studies* 6, 2 (2011), 75–89.

[51] Yichi Zhang and Joyce Chai. 2021. Hierarchical task learning from language instructions with unified transformers and self-monitoring. *arXiv preprint arXiv:2106.03427* (2021).

[52] Yu Zhong, TV Raman, Casey Burkhardt, Fadi Biadsy, and Jeffrey P Bigham. 2014. JustSpeak: enabling universal voice control on Android. In *Proceedings of the 11th Web for All Conference*. 1–4.

[53] Hong Zou and Jutta Treviranus. 2015. ChartMaster: A tool for interacting with stock market charts using a screen reader. In *Proceedings of the 17th International ACM SIGACCESS Conference on Computers & Accessibility*. 107–116.

A Appendix

A.1 Example Prompt for the Seed LLM

Comment: Diverse Applications Few-shot guiding examples
App: File Explorer
Control-Value pair: (new item, folder)
Response: Create a new folder.

App: Wordpad
Control-Value pair: (strikethrough, none)
Response: Strike the selected text.

App: Zoom
Control-Value pair: (start a new meeting with video off, none)
Response: Start a new meeting without the video.

App: Notepad
Control-Value pair: (edit, cut)
Response: cut the selected text

Comment: Generating more examples
App: Word
Control-Value pair: (share, copy link)

Comment: LLM-generated output
Response: Share the document as a link.

A.2 Example Prompt for the Gen LLM

Comment: Relevant Few-shot guiding example
Command: Change the Margin to Narrow.
Response: (Margins, Narrow)

Command: Change the Margin to Normal.
Response: (Margins, Normal)

Command: Change the Margin to Moderate.
Response: (Margins, Moderate)

Command: Change the Margin to Wide.
Response: (Margins, Wide)

Command: Change the Margin to Mirrored.
Response: (Margins, Mirrored)

Comment: User request
User: Set the Margin to Narrow.

Comment: LLM-generated output
Response: (Margins, Narrow)

A.3 Wizard-of-Oz-study

A.3.1 ***Participants.*** We recruited 11 ($P1$ to $P11$) screen reader users for our study. The average age was 52.5 (median=58, SD=15.1, range=31-73). Gender representation was almost equal, with 6 Female and 5 male participants. Recruitment was done using an email list with simple random sampling. Inclusion criteria for the study were: (i) exclusive dependence on a screen reader with no functional vision, (ii) proficiency with *JAWS* screen reader software, and (iii) familiarity with *Windows* operating system platform and desktop applications.

Participants were asked about their voice assistant usage habits during a pre-study interview. Almost all of the participants (10) mentioned that they used voice assistant systems at least once a day. $P2$ said he uses them at least once a week. 8 participants mentioned that they use use Siri [3], 9 participants mentioned Amazon Alexa [2], and 4 participants said they use Google Assistant [18]. Some common use cases for voice assistants were setting up alarms/timers/reminders, dictating messages and emails, making phone calls, or querying information such as weather, traffic, or trivia. More details about participant demographics and their voice assistant usage habits can be found in Table 3.

A.3.2 ***Design.*** During the study, participants were given tasks from 5 applications: *Excel*, *File Explorer*, *Gmail*, *Word*, and *Zoom*. These tasks can be found in Table 4. In total, there were 26 tasks to complete. The selection of these tasks was based on a closely related work [46], and these tasks were comparable in difficulty and captured the diversity in UI interactions for screen reader users.

A.3.3 ***Procedure.*** The experimenter first explained to the participants that the goal of the study was to collect the variations of natural language commands used to perform the task. The study was set up in a way that made it hard for them to discern that they were interacting with the wizard rather than the system. The experiment was conducted using a *Windows* laptop with a JAWS screen reader. During the study, the users leaned forward as if to speak into the microphone and only turned toward the experimenter when they had questions. Finally, many users expressed interest in getting a copy of the system for testing as soon as possible, which also suggests that they preferred interacting with a functional voice-enabled interface.

Since voice-enabled interaction with the desktop applications was a new concept to all the participants, the experimenter provided a demonstration at the beginning of the experiment by performing a simple task and showing a wide range of voice commands that the participants could issue. After the demo session and collection of demographic data, participants were given 10 minutes to learn and get familiar with the voice-enabled user interface. After the practice and learning stages, participants completed tasks in 5 applications. The order of the applications was counterbalanced using the Latin square method [8]. Each study lasted for nearly 1 hour, and the participants were given monetary compensation for their time and contribution.

A.3.4 ***Data collection.*** Audio recordings of the participants were collected during the study. These recordings were transcribed, and the participants' natural language commands for each task were collected.

[8] http://compneurosci.com/wiki/images/9/98/Latin_square_Method.pdf

ID	Gender	Age	Acuity	Voice Assistant	Use Cases
P1	Male	40	No vision	Alexa and Siri	Calendar, Email, Messages, and Reminder
P2	Male	58	No vision	GA	Calls, Email, and Messages
P3	Female	64	Light perception	Alexa and Siri	Info Query and Timers
P4	Female	61	No vision	Siri	Alarms, Calls, Email, Info Query, and Messaging
P5	Female	39	No vision	Alexa, GA, and Siri	Calculation, Games, and Info Query
P6	Female	35	Light perception	Alexa and Siri	Alarms, Calling, Emails, Start Apps, and Timers
P7	Male	43	Light perception	Alexa, GA, and Siri	App Navigation, Info Query, and Online Shopping
P8	Male	71	No vision	Alexa and Siri	Calls, Calculation, and Contacts
P9	Female	73	No vision	Alexa	Calculations and Info Query
P10	Male	62	Light perception	Alexa and GA	Alarms, Online Shopping, Smart Home, and Timers
P11	Female	31	Light perception	Alexa and Siri	Calls, Info Query, Online Shopping, and Timers

Table 3: Wizard-of-Oz study participant demographics. GA stands for Google Assistant. All information was self-reported.

Application	Task
Excel	Add a sum function to this cell
Excel	Freeze the top row
Excel	Change the format of this number to fraction
Excel	Highlight the selected cell yellow
File Explorer	Move dog picture to photos folder
File Explorer	Create a new folder named "Tasks"
File Explorer	Sort files in folder
File Explorer	Select all files
File Explorer	Share selected files
File Explorer	Open file with Adobe Acrobat
Gmail	Find the email from Google
Gmail	Move email to promotions folder
Gmail	Go to starred emails folder
Gmail	Reply to email saying "Thank you"
Gmail	Refresh your inbox
Word	Grow font size to 14
Word	Make text bold
Word	Center the text
Word	Undo action
Word	Switch style to heading 1
Word	Add a comment saying "change later"
Word	Insert a table with 2 rows and 3columns
Zoom	start a new meeting with video off
Zoom	Copy the invitation link to the meeting
Zoom	Schedule a meeting for tomorrow 1 PM
Zoom	Message your contact Alex saying "Hello"

Table 4: Tasks given to the participants in the Wizard-of-Oz study.

A.4 List of Applications used in the few-shot examples dataset

Application
Excel
Explorer
Notepad
Outlook
Spotify
Winword
Wordpad
Zoom
Google Docs (Opened in Chrome)
Amazon (Opened in Firefox)
Gmail (Opened in Chrome)

Table 5: List of Applications used in the few-shot examples dataset.